



AI ASSISTED CODE REVIEW SYSTEM

Student Details:

1. H. PRAVALIKA(213J1A0565)
2. G. SUNIL KUMAR(213J1A0551)
3. A. TARUNI(213J1A0501)
4. G. PRENITH SAI(213J1A0563)

Project Guide:

Dr. P. Appala Naidu

Abstract:

With the growing complexity of modern software systems, manual code review process is time-consuming and can lead to inconsistencies and missed opportunities for code quality improvements, especially in large codebases. So Automated tools for code review have become increasingly necessary to assist developers in maintaining high code quality and enforcing best practices. In this project we propose an enhanced AI assisted code review system that uses longer transformer models and adaptive learning methods to handle larger codebases and for real time developer feedback, allowing it to stay up-to-date with evolving best practices. This aims to offer a more comprehensive, scalable and developer friendly solutions.

Literature Survey

1.”An Automatic System for Modern Code Review”

Published On: ScienceDirect

Year: 2024

Link: <https://www.sciencedirect.com/science/article/pii/S2352711024000487>

Algorithms Used	Working	Drawback
1. Static Code Analysis 2. Rule-Based Systems 3. Basic matching learning models(for pattern matching and simple guideline enforcement)	It automatically detects simple issues like formatting, naming convention, and basic rule violations by integrating into modern CI/CD pipelines to flag issued before the code is merged	Limited capacity to handle more nuanced problems that require human judgement, such as code quality improvements or optimization suggestions.

2. “A CASE STUDY ON MACHINE LEARNING MODEL FOR CODE REVIEW EXPERT SYSTEM”

Published By: Vishwesh Kumar, ResearchGate
Year:2017

Link:

https://www.researchgate.net/publication/320012714_A_case_study_on_machine_learning_model_for_code_review_expert_system_in_software_engineering

Algorithms Used	Working	Drawback
- Supervised Learning(Support Vector Machines, Random Forests). - Static analysis techniques. - Hybrid approach combining ML and rule-based analysis.	It uses machine learning to predict bugs and issues based on patterns in previous reviews. The use of machine learning allows the system to go beyond simple rule-based analysis and detect more complex issues.	The ML models struggle with novel code structures or frameworks that were not in the training data, requiring additional manual review.

3. “DEEP LEARNING APPROACH FOR CODE REVIEW AUTOMATION”

Published By: International Journal of Science and Research(IJSR)
Year:2024

Link: <https://www.ijsr.net/archive/v12i2/SR24716230727.pdf>

Algorithms Used	Working	Drawback
1. Sequence-to-Sequence models. 2. Transformer neural networks. 3. Deep neural networks(for code diff analysis)	It is a deep learning-based approach which improves the system’s ability to detect more sophisticated issues, such as logical inconsistencies or code smells, by understanding patterns in the code.	High computational cost and longer model training times. The model is prone to overfitting and may require large datasets for effective generalization.

4. “A MACHINE LEARNING SYSTEM FOR LARGE-SCALE CODE REVIEW IN INDUSTRY”

Published By: MDPI, Electronics
Year:2023

Link: <https://www.mdpi.com/2079-9292/13/4/767>

Algorithms Used	Working	Drawback
1. Supervised and Unsupervised learning. 2. Clustering algorithms. 3. Classification techniques. 4. Hybrid machine learning systems for continuous systems.	It is a deep learning-based approach which improves the system's ability to detect more sophisticated issues, such as logical inconsistencies or code smells, by understanding patterns in the code.	High computational cost and longer model training times. The model is prone to overfitting and may require large datasets for effective generalization.

5. “AUTOMATING CODE REVIEWS USING LARGE LANGUAGE MODELS”

Published By: ArXiv

Year:2019

Link: <https://arxiv.org/pdf/1912.09652>

Algorithms Used	Working	Drawback
1. Large Language Models(LLMs) 2. Transformer models 3. Pre-trained models fine-tuned on code review datasets	It uses LLMs like GPT to generate comments and suggestions for code reviews. These models are trained on large datasets of code and code reviews to understand code semantics and suggest improvements.	LLMs can be computationally expensive and may introduce false positives if not fine-tuned adequately.

6. “AI-ASSISTED ASSESSMENT OF CODING PRACTICES IN MODERN CODE REVIEW”

Published By: Google, University of Washington
Year:2024

Link:
https://homes.cs.washington.edu/~rjust/publ/code_review_automation_aiware_2024.pdf

Algorithms Used	Working	Drawback
1. Large Language Models(LLMs) 2. Transformer based models(T5) 3. Large-scale preprocessing for code review data	This AutoCommenter system developed uses transformer models to automate the detection process across several programming languages. It integrates into both IDEs and code review systems to provide real time feedback to developers.	• Context limitations • Outdated practices • Limited diversity of suggestions.

Proposed System:

Our proposed system aims to automate code reviews more effectively by overcoming the drawbacks like context limitations, code repairs and suggestion generations etc. which found in existing systems.

Our system approaches with the following benefits:

1. Improved context window using newer models such as Transformer-XL or Longformer.
2. By incorporating Reinforcement learning(RL), the system could dynamically adapt and improve based on user feedback in real time.
3. Introducing OpenAI for full functional implementation of code repairs and suggestion generations.

Conclusion:

In conclusion, we propose an advanced AI code review system that addresses the limitations of existing models and provides deeper insights into code logic and control flow, ensuring complex bugs are effectively identified.

This code review tool will not only improve the accuracy and speed of the review process but also enhance developer productivity and maintainability of code over time. It provides a smarter, more scalable, and context aware solution for modern software development challenges.